SENATI

Escuela de Tecnologías de la Información

SOLUCIONARIO

Solucionario Guía 11

Redes Neuronales Artificiales: Estructura y Creación

Solución paso a paso de los ejercicios propuestos y de las preguntas de reflexión

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Curso	Seminario de Complementación Práctica I (PIAD-427)
Semana	11
Documento	Solucionario de los ejercicios propuestos de la Guía 11
Herramientas	Python 3 · Visual Studio Code · TensorFlow · Keras · NumPy
Instructor	Rembrandt Ubalde Enríquez
Uso	Material del instructor

Contenido

Indicaciones generales 3
Ejercicio 1. Perceptrón que aprende la compuerta AND 4
Paso 1. Entender el problema 4
Paso 2. La regla de aprendizaje del perceptrón 4
Paso 3. Programa 5
Paso 4. Explicación 5
Ejercicio 2. Parámetros de la red 4 – 8 – 6 – 3 7
Paso 1. Cálculo a mano 7
Paso 2. Verificación con Keras 7
Ejercicio 3. Comparar funciones de activación 8
Paso 1. Programa 8
Paso 2. Análisis 8
Ejercicio 4. Agregar una segunda capa oculta 10
Paso 1. Nueva estructura 10
Paso 2. Programa 10
Paso 3. Respuesta explicada 10
Ejercicio 5. XOR en TensorFlow Playground 12
Paso 1. ¿Por qué XOR es difícil? 12
Paso 2. Configuraciones de referencia 12
Paso 3. Explicación 12
Respuestas a las preguntas de reflexión 13

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Indicaciones generales

Cada ejercicio se resuelve en un archivo nuevo dentro de la carpeta **IA_Guia11_Redes_Neuronales**, con el entorno virtual activo. Los nombres de los archivos empiezan con `ej` seguido del número del ejercicio.

Ejercicio	Archivo	Librerías
1	`ej01_perceptron_and.py`	NumPy
2	`ej02_parametros.py`	TensorFlow / Keras
3	`ej03_activaciones.py`	TensorFlow / Keras, scikit-learn
4	`ej04_dos_ocultas.py`	NumPy
5	Sin código	Navegador: playground.tensorflow.org

Los resultados de los ejercicios 1, 2 y 4 son **exactos** (no dependen del azar). En el ejercicio 3 el entrenamiento de la red produce valores que pueden variar en algunos decimales según la versión de TensorFlow y el procesador; por eso su resultado se presenta como **aproximado**.

Ejercicio 1. Perceptrón que aprende la compuerta AND

Enunciado: Programa con NumPy un perceptrón (una neurona con función escalón) que aprenda la compuerta lógica AND usando la regla de aprendizaje del perceptrón. Muestra los pesos finales y la salida para las 4 combinaciones.

Paso 1. Entender el problema

x1	x2	AND (x1 y x2)
0	0	0
0	1	0
1	0	0
1	1	1

La neurona calcula `z = x1·w1 + x2·w2 + b` y aplica la **función escalón**: devuelve 1 si z ≥ 0 y 0 en caso contrario.

Paso 2. La regla de aprendizaje del perceptrón

Regla de actualización

```
error = valor_esperado - valor_predicho        (puede ser -1, 0 o 1)
w = w + tasa · error · x
b = b + tasa · error

Si el error es 0, los pesos no cambian.
```

Paso 3. Programa

Archivo: ej01_perceptron_and.py

```
# ej01_perceptron_and.py - Perceptrón que aprende la compuerta AND
import numpy as np

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])   # entradas
y = np.array([0, 0, 0, 1])                       # salida AND

w = np.zeros(2)          # pesos iniciales
b = 0.0                  # sesgo inicial
tasa = 0.1               # tasa de aprendizaje

def escalon(z):
    return 1 if z >= 0 else 0

for epoca in range(1, 21):
    errores = 0
    for xi, objetivo in zip(X, y):
        pred = escalon(np.dot(xi, w) + b)
        error = objetivo - pred
        if error != 0:                           # regla del perceptrón
            w += tasa * error * xi
            b += tasa * error
            errores += 1
    print(f"Época {epoca}: errores = {errores} | w = {w.round(2)} | b = {b:.2f}")
    if errores == 0:
        print("¡Aprendió la compuerta AND!")
        break

print("\nx1 x2 | AND | perceptrón")
for xi, objetivo in zip(X, y):
    print(f" {xi[0]}  {xi[1]} |  {objetivo}  |     {escalon(np.dot(xi, w) + b)}")
```

▶ Resultado esperado en la terminal

```
Época 1: errores = 2 | w = [0.1 0.1] | b = 0.00
Época 2: errores = 3 | w = [0.2 0.1] | b = -0.10
Época 3: errores = 3 | w = [0.2 0.1] | b = -0.20
Época 4: errores = 0 | w = [0.2 0.1] | b = -0.20
¡Aprendió la compuerta AND!

x1 x2 | AND | perceptrón
 0  0 |  0  |     0
 0  1 |  0  |     0
 1  0 |  0  |     0
 1  1 |  1  |     1
```

Paso 4. Explicación

- El perceptrón empieza con pesos en 0 y se equivoca. En cada error corrige sus pesos un poco (tasa = 0.1).
- En la **época 4** ya no comete errores: encontró **w1 = 0.2, w2 = 0.1, b = -0.2**.
- Comprobación con (1, 1): z = 0.2 + 0.1 - 0.2 = 0.1 ≥ 0 → **1**. Con (0, 1): z = 0.1 - 0.2 = -0.1 < 0 → **0**.
- Caso interesante (1, 0): en papel z = 0.2 - 0.2 = 0, pero Python guarda el sesgo como -0.20000000000000004 por la forma en que almacena los decimales; z queda en un negativo diminuto (-2.8e-17) y la salida es **0**, que es lo correcto. Puedes verlo con `print(repr(b))`.

- El perceptrón es la red neuronal más simple: **una sola neurona**. Puede aprender AND y OR porque se separan con una recta, pero **no puede aprender XOR** (ver Ejercicio 5).

Ejercicio 2. Parámetros de la red 4 – 8 – 6 – 3

Enunciado: Calcula a mano el número de parámetros de una red 4 – 8 – 6 – 3 y verifica tu respuesta creando el modelo en Keras con `count_params()`.

Paso 1. Cálculo a mano

En una capa densa: **parámetros = entradas × neuronas + neuronas (sesgos)**.

Capa	Entradas	Neuronas	Pesos	Sesgos	Parámetros
Oculta 1	4	8	4 × 8 = 32	8	**40**
Oculta 2	8	6	8 × 6 = 48	6	**54**
Salida	6	3	6 × 3 = 18	3	**21**
Total			98	17	**115**

> ⚠ **Atención:** La capa de entrada **no tiene parámetros**: solo recibe los datos. Un error común es sumar también «4» por la capa de entrada.

Paso 2. Verificación con Keras

Archivo: ej02_parametros.py

```
# ej02_parametros.py - Verificar con Keras los parámetros de la red 4-8-6-3
from tensorflow import keras

modelo = keras.Sequential([
    keras.Input(shape=(4,)),
    keras.layers.Dense(8, activation="relu"),
    keras.layers.Dense(6, activation="relu"),
    keras.layers.Dense(3, activation="softmax"),
])
for capa in modelo.layers:
    print(f"{capa.name:<8} parámetros = {capa.count_params()}")
print("Total de parámetros:", modelo.count_params())
```

▶ Resultado esperado en la terminal

```
dense    parámetros = 40
dense_1  parámetros = 54
dense_2  parámetros = 21
Total de parámetros: 115
```

Respuesta: la red tiene **115 parámetros**, igual que el cálculo a mano. Los nombres `dense`, `dense_1` y `dense_2` los pone Keras automáticamente; si ejecutas el script varias veces en el mismo programa pueden cambiar a `dense_3`, `dense_4`, etc.

Ejercicio 3. Comparar funciones de activación

Enunciado: En la Práctica 04 cambia la activación de las capas ocultas de `relu` a `tanh` y luego a `sigmoid`. Compara la pérdida final y la exactitud en prueba. ¿Cuál aprende más rápido?

Paso 1. Programa

En lugar de editar la Práctica 04 tres veces, un bucle `for` crea y entrena la misma red con cada activación. La semilla se fija **dentro** del bucle para que las tres redes empiecen en igualdad de condiciones.

Archivo: ej03_activaciones.py

```
# ej03_activaciones.py - Comparar relu, tanh y sigmoid en la red de la Práctica 04
from tensorflow import keras
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
esc = StandardScaler()
X_tr, X_te = esc.fit_transform(X_tr), esc.transform(X_te)

print(f"{'Activación':<10}{'Pérdida época 25':>18}{'Pérdida final':>15}{'Exactitud
prueba':>18}")
for act in ["relu", "tanh", "sigmoid"]:
    keras.utils.set_random_seed(42)
    modelo = keras.Sequential([
        keras.Input(shape=(4,)),
        keras.layers.Dense(10, activation=act),
        keras.layers.Dense(8, activation=act),
        keras.layers.Dense(3, activation="softmax"),
    ])
    modelo.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
    h = modelo.fit(X_tr, y_tr, epochs=100, batch_size=8, verbose=0)
    _, acc = modelo.evaluate(X_te, y_te, verbose=0)
    print(f"{act:<10}{h.history['loss'][24]:>18.4f}{h.history['loss'][-
1]:>15.4f}{acc:>18.2%}")
```

▶ Resultado aproximado (los decimales pueden variar en tu PC)

```
Activación  Pérdida época 25  Pérdida final  Exactitud prueba
relu                 0.3015         0.0587            96.67%
tanh                 0.2330         0.0590            96.67%
sigmoid              0.9870         0.3410            93.33%
```

Paso 2. Análisis

Activación	Velocidad de aprendizaje	Motivo
ReLU	Rápida	Su derivada es 1 para valores positivos: el error se propaga bien hacia atrás.
Tanh	Rápida (incluso un poco más al inicio)	Sus salidas están centradas en 0 (de -1 a 1), lo que ayuda al optimizador.

Activación	Velocidad de aprendizaje	Motivo
Sigmoide	**Lenta**	Su derivada es como máximo 0.25: el error se hace pequeño al retroceder por las capas (**desvanecimiento del gradiente**).

Respuesta: ReLU y tanh aprenden más rápido y alcanzan alrededor de 97 % de exactitud. Con sigmoide la pérdida a las 100 épocas sigue alta (≈ 0.34); necesitaría muchas más épocas. Por eso **ReLU** es la opción por defecto para las capas ocultas y la sigmoide se reserva para la **salida** de problemas binarios.

Ejercicio 4. Agregar una segunda capa oculta

Enunciado: Modifica la Práctica 03 para agregar una segunda capa oculta de 3 neuronas con activación tanh. ¿Cuántos parámetros tiene ahora la red? ¿Qué clase predice?

Paso 1. Nueva estructura

Red 3 – 4 – 3 – 2

```
Entrada (3) --W1 3x4--> Oculta 1 (4, ReLU) --W2 4x3--> Oculta 2 (3, tanh) --W3 3x2--> Salida (2, softmax)
```

Cambia la forma de W2: antes unía la oculta con la salida (4 × 2); ahora une la oculta 1 con la nueva oculta 2 (**4 × 3**). Se agrega W3 (**3 × 2**) hacia la salida.

Paso 2. Programa

Archivo: ej04_dos_ocultas.py

```
# ej04_dos_ocultas.py - Red 3-4-3-2: se agrega una segunda capa oculta con tanh
import numpy as np

np.random.seed(1)
W1 = np.random.randn(3, 4).round(2); b1 = np.zeros(4)   # entrada  -> oculta 1
W2 = np.random.randn(4, 3).round(2); b2 = np.zeros(3)   # oculta 1 -> oculta 2 (NUEVA)
W3 = np.random.randn(3, 2).round(2); b3 = np.zeros(2)   # oculta 2 -> salida

def relu(z): return np.maximum(0, z)
def softmax(z):
    e = np.exp(z - z.max())
    return e / e.sum()

x = np.array([0.5, 0.1, 0.9])
h1 = relu(x @ W1 + b1)
h2 = np.tanh(h1 @ W2 + b2)          # segunda capa oculta con tanh
y = softmax(h2 @ W3 + b3)

print("Oculta 1 (ReLU):", h1.round(3))
print("Oculta 2 (tanh):", h2.round(3))
print("Probabilidades de salida:", y.round(3))
print("Clase predicha:", y.argmax())
p = [W1.size + b1.size, W2.size + b2.size, W3.size + b3.size]
print(f"Parámetros por capa: {p} -> total = {sum(p)}")
```

▶ Resultado esperado en la terminal

```
Oculta 1 (ReLU): [1.185 0.    1.223 0.   ]
Oculta 2 (tanh): [-0.319  0.253 -0.006]
Probabilidades de salida: [0.428 0.572]
Clase predicha: 1
Parámetros por capa: [16, 15, 8] -> total = 39
```

Paso 3. Respuesta explicada

- **Parámetros:** (3·4 + 4) + (4·3 + 3) + (3·2 + 2) = 16 + 15 + 8 = **39** (antes eran 26).

- La primera capa oculta da el **mismo resultado** que en la Práctica 03 porque W1 se genera igual con la semilla 1.
- La segunda capa usa **tanh**, por eso sus valores pueden ser negativos (-0.319).
- La red ahora predice la **clase 1** (57.2 %), mientras que la red de la Práctica 03 predecía la clase 0. Como los pesos son aleatorios, cambiar la estructura cambia la predicción: una red **sin entrenar** no da respuestas confiables.

Ejercicio 5. XOR en TensorFlow Playground

Enunciado: En TensorFlow Playground, con el conjunto de datos XOR, encuentra la red más pequeña (menos neuronas) que logre un Test loss menor a 0.05. Registra la configuración y explica el resultado.

Paso 1. ¿Por qué XOR es difícil?

En XOR los puntos azules están en dos cuadrantes opuestos y los naranjas en los otros dos. **No existe una recta** que los separe, por eso un perceptrón o una red **sin capas ocultas** fracasa (Test loss cercano a 0.5).

Paso 2. Configuraciones de referencia

Entradas (FEATURES)	Capas ocultas	Activación	Resultado esperado
X1, X2	Ninguna	—	No aprende (Test loss ≈ 0.5).
X1, X2	1 capa de **2** neuronas	Tanh	A veces aprende, a veces se estanca: depende de los pesos iniciales.
X1, X2	1 capa de **4** neuronas	Tanh o ReLU	Test loss < 0.05 en unas 300 a 800 épocas (configuración pequeña y estable).
X1X2 (producto)	Ninguna	—	Test loss < 0.05 casi de inmediato: **0 neuronas ocultas**.

Configuración usada: Learning rate **0.03**, Ratio of training data **50 %**, Noise **0**, Batch size **10**. Los valores exactos de pérdida cambian en cada ejecución porque el simulador inicializa los pesos al azar; se aceptan respuestas con 3 a 4 neuronas ocultas.

Paso 3. Explicación

- Con solo X1 y X2 se necesita **al menos una capa oculta**: cada neurona oculta traza una recta y la capa de salida las combina para formar las regiones del XOR.
- Con la característica **X1X2** el problema se vuelve lineal: X1·X2 es positivo en dos cuadrantes y negativo en los otros dos, justo lo que separa XOR. Una buena **característica de entrada** puede reemplazar neuronas.
- Conclusión: las capas ocultas permiten que la red **cree sus propias características**; esa es la gran ventaja del Deep Learning.

Respuestas a las preguntas de reflexión

1. ¿Por qué una red neuronal necesita funciones de activación no lineales?

Porque sin ellas cada capa solo hace una multiplicación y una suma, y una combinación de operaciones lineales **sigue siendo lineal**. Una red de 10 capas sin activación equivale a una sola capa y solo trazaría rectas. Las funciones no lineales (ReLU, tanh, sigmoide) permiten aprender **curvas** y patrones complejos, como se vio en la Práctica 05 con las dos lunas.

2. ¿Qué diferencia hay entre los pesos y los hiperparámetros?

Los **pesos** y sesgos son los parámetros que la red **aprende sola** durante `fit()` (por ejemplo, los 165 parámetros de la Práctica 04). Los **hiperparámetros** los elige **la persona** antes de entrenar: número de capas y neuronas, funciones de activación, épocas, tamaño de lote, optimizador y tasa de aprendizaje.

3. ¿Qué tipo de red usarías en cada caso?

- **a) Demanda eléctrica de la próxima hora:** red **recurrente (RNN/LSTM)**, porque los datos son una **serie de tiempo** y la demanda depende de las horas anteriores.
- **b) Placas de autos en una foto:** red **convolucional (CNN)**, porque trabaja con **imágenes** y sus filtros detectan formas y caracteres.
- **c) Aprobar un préstamo:** red **feedforward (FNN)**, porque los datos son una **tabla** (ingresos, edad, deudas) sin orden temporal ni estructura de imagen.

4. ¿Por qué muchas capas ocultas pueden empeorar el resultado?

Porque la red tiene tantos parámetros que puede **memorizar** los datos de entrenamiento, incluido su ruido, en lugar de aprender el patrón general: es el **sobreajuste**. Su exactitud en entrenamiento sube, pero en datos nuevos baja. Además, las redes muy profundas tardan más en entrenar y pueden sufrir el desvanecimiento del gradiente. Se recomienda empezar con pocas capas y agregar más solo si la exactitud en prueba mejora.